

FlagOS 超长上下文数据标注挑战赛 —— UFU 团队技术报告

团队名称： UFU

一、问题定义与总体思路

1.1 竞赛要求

FlagOS 超长上下文数据标注挑战赛要求参赛团队使用 Qwen3-4B 开源模型，在 30K token 的上下文窗口内（Task 8 为 16K），利用 In-Context Learning（ICL）完成 8 个异构任务的自动标注。每个任务包含数百至数千条训练示例和 500 条测试样本（Task 8 为 166 条）。

Task	任务名称	任务类型	训练示例数	测试样本
1	closest_integers	数值计算（最小绝对差）	5,500	500
2	count_nouns_verbs	词性计数（名词/动词）	5,440	500
3	collatz_conjecture	数值变换（Collatz 规则）	3,997	500
4	conala_concat_strings	字符串拼接	3,993	500
5	semeval_tweet_sadness	情感分类（Sad/Not sad）	1,899	500
6	mnli_same_genre	NLI 推理（Y/N）	5,500	500
7	jeopardy_answer	知识问答（Jeopardy 线索）	5,499	500
8	kernel_generation	代码生成（Triton 核函数）	184	166

1.2 核心矛盾

这个竞赛面临的挑战可以归结为三个层面的矛盾：

矛盾一：训练示例数量远超上下文容量，选错示例不仅浪费空间，更会引入错误模式

任务	训练示例数	30K 可容纳约	溢出倍数
T1/T2/T6/T7	5,400~5,500	~300	18×
T5	1,899	~300	6×
T8	184	~100	2×（16K窗口）

这不是简单的"装不下"的问题。选错的示例不仅浪费了有限的 token 预算，更重要的是——它们会给模型展示错误的推理模式。例如在 T4（字符串拼接）中，如果选入的示例输出恰好形成了常见英文单词（如 the），模型会误以为拼接需要添加空格，从而在所有测试样本上犯同样的错误。因此，示例选择不是"越多越好"，而是"越准越好"。

矛盾二：8 个任务横跨完全不同的推理模式，一套提示词无法通吃

- T1/T3 是确定性计算：答案唯一，规则明确，ICL 后可达 99%+
- T4 是确定性操作：答案唯一，但模型容易产生幻觉
- T5/T6 是二分类：答案空间小，但边界模糊，需要模型理解微妙的情感/语义差异
- T7 是开放知识问答：答案空间极大，需要广泛的外部知识
- T8 是代码生成：需要理解特定编程框架

矛盾三：4B 小模型在复杂任务上的能力存在固有限制，不是提示词工程能完全解决的

为了量化这一限制，我们在竞赛初期对部分任务进行了 few-shot baseline 测试（用前 100 条训练示例作为 ICL），并对比了 qwen3.6-plus（更大规模的商业模型）的 zero-shot 能力：

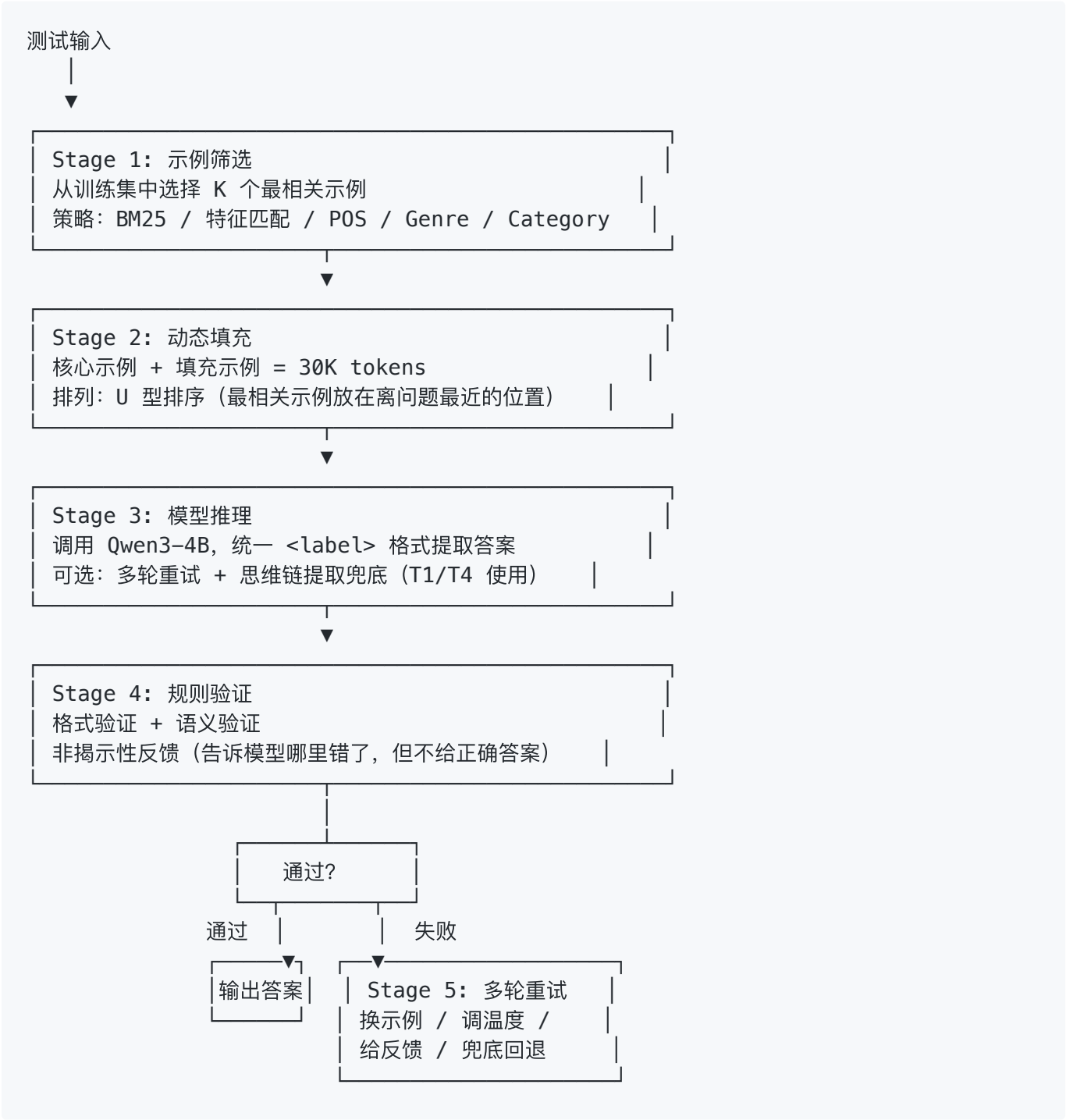
任务	Few-shot baseline (Qwen3-4B)	ICL 最优 (Qwen3-4B)	qwen3.6-plus (Zero-shot)
T1 最近整数对	77.6%	100%	-
T3 Collatz 变换	21.4%	100%	-
T4 字符串拼接	30%	95.6%	-
T7 Jeopardy 问答	36%	~43%	80%

Few-shot baseline 在 T1/T3/T4 上的正确率远低于 ICL 最优版本，说明 4B 模型在简单 ICL 设置下远未发挥全部潜力，需要通过精细的示例选择和推理策略才能接近上限。而对于 T7 这样的知识密集型任务，即使有 ICL 引导，4B 模型也只能达到 ~43%，而 qwen3.6-plus 在 zero-shot 下

就能取得 80%——这证明了模型规模对知识密集型任务的决定性影响，不是提示词设计能弥补的。

1.3 总体解决方案：五阶段流水线

基于上述矛盾分析，我们设计了 "筛选 → 填充 → 推理 → 验证 → 重试" 五阶段流水线框架：



框架的两个关键设计原则：

1. **任务独立封装，统一推理接口**：每个任务作为 `BaseTask` 的子类，独立实现筛选策略、验证规则、重试逻辑。但所有任务共享同一个 `run_inference()` 入口，确保实验可比、模块可复用。
2. **每阶段的策略因任务而异**：筛选不是统一的 BM25，填充不是简单的追加，验证不是简单的格式检查——每个阶段的策略都针对任务特性定制。

二、各任务深入讨论

接下来针对每个任务，按照"任务难点 → 我们的方案 → 为什么这样做 → 数据证明"的逻辑展开。

2.1 Task 1：最近整数对

任务：给定整数列表，求最小绝对差。

难点：模型需要理解"最小绝对差"的数学概念，在包含负数的列表中保持稳定，且答案分布高度偏斜（~~41%~~的答案为-13，~20% 为 0）。

我们的方案：特征匹配检索 + 答案桶均衡

第一步，从输入列表提取特征（长度、值域范围、是否含负数、是否含重复），计算与训练示例的特征相似度，取 top-100 最相似示例。

第二步，将这 100 条示例按答案值分桶：

桶 0：答案 = 0	(约 20%)
桶 1：答案 = 1~3	(约 41%，最密集)
桶 2：答案 = 4~6	
桶 3：答案 = 7~15	
桶 4：答案 > 15	

从每个桶中各选 1 个最相似示例（共 5 个核心），确保每种答案范围都有代表。

为什么这样做：最初 v3 版本只用特征匹配，准确率从 96.4% 回退到 91.2%。原因是特征（长度、范围、是否含负数）的区分度不够——两条列表长度相同、范围相近的输入，答案可能完全不同。模型更需要的不是"输入相似的示例"，而是"覆盖各种答案值"的示例。答案桶均衡解决了这个问题。

数据证明：

版本	策略	正确数/500
v1（基线）	前 100 条	482
v3	纯特征匹配	456（回退）
v6	特征匹配 + 答案桶均衡	488
v8	特征匹配 + 答案桶 + 4轮重试	496

2.2 Task 2：词性计数

任务：给定英文句子，统计名词或动词的数量。

难点：名词判定规则复杂（复合名词 `fire hydrant` = 2，修饰名词 `train tracks` 中 `train` 也算）；动词判定涉及分词、过去分词、助动词；边界情况（0 和 5 的样本最少，2~3 最密集）。

我们的方案：POS 检索 + 精选示例 + 4 轮重试

第一步，判断测试输入是"数名词"还是"数动词"，只从相同类型的示例池中检索（POSRetriever）。

第二步，手工精选 10 条高质量示例，覆盖 `noun(05)` + `verb(05)` 的全部可能范围，按 U 型排列放在 prompt 头部和尾部（注意力焦点区域）。

第三步，多轮重试：R0（固定精选）→ R1（POS 动态检索）→ R2（反馈：如"超过总词数"）→ R3（提升温度）→ Fallback。

为什么这样做：如果测试输入是"数名词"，但检索到的示例都是"数动词"的，模型会学习到错误的计数模式。POS 检索确保任务类型一致。精选示例覆盖全部答案范围，避免模型在边界值（0 或 5）上出错。验证规则"不能超过句子总词数"是最有价值的——模型经常会数多。

数据：T2 从基线 ~70% 提升到 ~80%。

2.3 Task 3：Collatz 变换

任务：给定整数列表，对每个元素应用 Collatz 规则（偶÷2，奇×3+1）。

难点：这是 8 个任务中规则最明确的一个，唯一的难点是输出列表长度必须严格匹配输入。

我们的方案：长度匹配检索 + 数学一致性验证 + 思维链提取兜底

第一步，从训练集中找出与测试输入相同长度的所有示例，放在 prompt 最近端（注意力权重最高的区域）。

第二步，验证规则：长度必须匹配 + 数学一致性（偶数输入的输出必须小于输入，奇数输入的输出必须大于输入）。

第三步，如果三轮推理都失败，从模型的 `<think>` 思维链块中提取推理过程中的中间结果作为兜底答案。

为什么这样做：Collatz 是一一映射，相同长度的输入对应相同长度的输出。长度匹配让模型自然学会了一一对应的关系。数学一致性验证利用了 Collatz 规则的数学性质——偶 $\div 2$ 变小，奇 $\times 3+1$ 变大——不需要计算正确答案也能判断输出是否合理。思维链提取利用了模型在输出 `<label>` 之前通常会在 `<think>` 中逐步计算的事实。

数据证明：

版本	策略	正确数/500
v1	长度匹配	500
v8	长度匹配 + 验证反馈	499

2.4 Task 4：字符串拼接

任务：给定字符串列表，将所有字符串直接拼接（无分隔符）。

难点：拼接是确定性操作，但模型容易产生"幻觉"——当输出恰好形成常见英文单词（如 `the`，`and`）时，模型可能误以为需要添加空格。

我们的方案：特征匹配 + 清洁示例策略 + 5 轮重试 + 长度/字符验证

清洁示例策略：筛选出输出中不包含常见英文单词（`the`，`and`，`for` 等 60+ 个）的训练示例。这些示例的输出纯粹是字符拼接，没有意外形成有意义的单词，放在离问题最近的位置，避免模型产生"需要添加空格"的幻觉。

5 轮重试：

- R1：标准特征匹配
- R2：切换清洁示例
- R3：清洁示例 + 具体错误反馈 + `temperature=0.3`
- R4：去掉头部 5 个最相关清洁示例（避免过拟合最近端）+ `temperature=0.3`
- R5：替换头部 3 个 + `temperature=0.5`

验证规则：长度必须等于所有子串长度之和 + 输出字符频率必须等于输入字符频率之和。这两个验证覆盖 95% 以上的错误类型。

为什么这样做：v3 只用特征匹配时准确率从 77.6% 回退到 63.8%，因为只看输入相似性，不看输出质量，选入了大量"输出含常见单词"的干扰示例。v5 引入清洁示例 + 长度验证后，从 63.8% 提升到 86.8% (+23 %) ，证明**输出验证 + 示例质量过滤** 的组合比单纯改进检索更有效。

数据证明：

版本	策略	正确数/500
v1	BM25	388
v3	纯特征匹配	319（回退）
v5	清洁示例 + 长度验证	434
v6	+ 字符组成验证	472
v8	+ 5 轮重试	478

2.5 Task 5：推文情感检测

任务：判断推文是否表达 Sadness（Sad / Not sad）。

难点："Sad" 的定义在数据集中非常宽泛——不仅包括明确的悲伤表达，还包括轻微抱怨、吐槽、失望、负面评价。关键边界：带有 humor emoji 的 "unhappy and unfulfilled 🤔" 也算 Sad；"blues" 作为音乐类型不算 Sad。

我们的方案：BM25 检索 + 标签均衡 + 难度排序 + Emoji 移除

第一步，移除推文中的 emoji（避免干扰 BM25 检索），用 BM25 检索 top-40 最相关示例。

第二步，按 Sad / Not sad 标签均衡选择（各占一半），避免检索结果的类别偏差。

第三步，按难度倒序排列：困难示例（含否定词、转折词、矛盾情感）放在离问题最近的位置（注意力焦点），简单示例放在远端。

为什么这样做：简单示例（明确悲伤的推文）模型已经能正确判断，放在远端即可。困难示例（含否定词、转折词、矛盾情感）才是模型最需要学习的边界情况，放在最近端可以最大化模型对边缘情况的理解。标签均衡避免了检索结果中某一类别过多导致的预测偏斜。

数据：T5 从基线 ~80% 提升到 ~85-90%。

2.6 Task 6：NLI 推理

任务：给定前提和假设，判断假设是否被前提蕴含（Y/N）。

难点：NLI 需要深层语义理解，4B 模型容易受表面词汇匹配干扰。不同文本流派（fiction, telephone, slate, government）的语言风格差异巨大。

我们的方案：流派过滤 **BM25 + Y/N 标签均衡 + CoT 样例注入**

第一步，按 genre 构建独立的 BM25 索引（fiction 池、telephone 池、slate 池等）。检索时先匹配流派，再在流派内计算相似度。

第二步，从同流派中按 Y/N 标签均衡选择示例。

第三步，在 prompt 中注入少量 CoT（Chain-of-Thought）样例，展示推理过程。

为什么这样做：不同流派的文本风格差异巨大——fiction 包含文学性表达（"She gazed wistfully..."），telephone 包含口语化对话，government 包含正式公文语言。如果测试输入是 fiction，但检索到 telephone 风格的示例，模型可能无法正确推理。流派过滤是 T6 最大的单点优化。

数据证明：T6 从全局 BM25 的 ~50% 提升到流派过滤后的 ~70%（+20 个百分点），是所有检索策略中提升幅度最大的。

2.7 Task 7: Jeopardy 问答

任务：给定 Jeopardy 线索（Category + Clue），给出正确答案。

难点：这是 8 个任务中知识密度要求最高的一个。训练集（5499 条）中可能不包含与测试线索相同答案的示例，模型需要广泛的世界知识。

我们的方案：Category-First 检索 + 答案索引模糊匹配 + 两轮验证

第一步，先检索相同 Category 的示例放在 prompt 最近端，再用 BM25 示例补足。Category 定义了知识范围——"HISTORY" 的线索答案通常是历史人物或事件，"SCIENCE" 的通常是科学概念。

第二步，将第一轮答案在训练集中进行模糊匹配（支持单复数、所有格、名称截断、拼写相似度 ≥ 0.85 ），找到相同答案的示例。

第三步，如果找到匹配示例 → 构建验证 prompt 让模型确认；如果未找到 → 触发重新推理。

为什么这样做：对于知识问答，验证答案是否正确很难（需要外部知识），但验证"训练集中是否有相同答案"很容易。如果有，说明答案有先例，可以确认；如果没有，说明模型可能给出了训练集中不存在的冷门答案，需要重新推理。

数据证明：

版本	策略	正确率
zero-shot	zero-shot	~36%
v7	Category-First + 答案索引 + 两轮验证	~43%

2.8 Task 8: Triton 核函数生成

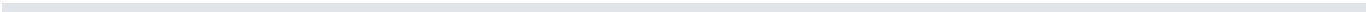
任务：给定功能描述，生成完整的 Triton GPU 核函数代码。

难点：训练集仅 184 条（远少于其他任务），16K token 也难以全部容纳。Triton 编程模型特殊（`tl.program_id`、`tl.load + mask`），4B 模型不熟悉。

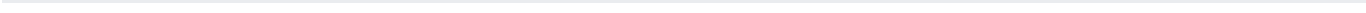
我们的方案：操作类型匹配 + 错误规则注入

第一步，用关键词（softmax, matmul, layernorm 等）匹配训练示例和测试输入的操作类型，相同类型的示例放在最近端。

第二步，在 prompt 中显式列出常见 Triton 错误（如 `tl.pi` 不存在、`tl.constexpr` 滥用等），帮助模型避免。



三、关键策略的横向总结



3.1 示例选择策略总览

策略	适用任务	核心思想
特征匹配	T1, T3, T4	输入结构相似 → 输出模式相似
答案桶均衡	T1	覆盖输出值分布，不只看输入相似
POS 检索	T2	任务类型（名词/动词）必须一致
长度匹配	T3	一一映射任务，长度是最强特征
清洁示例	T4	输出质量比输入相似更重要
标签均衡	T5, T6	分类任务避免类别偏差
难度排序	T5	困难示例放最近端，学边界情况
流派过滤	T6	同流派 = 同语言风格
Category-First	T7	Category 定义知识范围
操作类型匹配	T8	训练集小，按操作类型分组即可

3.2 验证规则设计原则

- 1. 非揭示性：告诉模型"哪里错了"。迫使模型重新思考。
- 2. 具体化：错误信息越具体，重试成功率越高。T4 不仅说"长度不对"，还列出每个子串的长度。
- 3. 利用任务数学性质：T3 的"偶÷2 变小、奇×3+1 变大"——不计算正确答案也能判断合理性。
- 4. 保守性：宁可误杀（把正确判错）也不放过（把错误判对）。误杀还有重试，放过就直接输出了。

3.3 多轮重试策略

任务	重试轮数	核心策略	收益
T1	4	标准→反馈(去零示例)→展示计算→高温+思维链提取	+0.7 分
T2	4	固定→动态→反馈→高温	+3~5 分
T3	3+CoT	长度匹配→反馈→思维链提取	+0.2 分
T4	5	换清洁示例→反馈→去头部→换+高温	+5~8 分
T5	2	BM25→反馈	+2.5 分
T7	2	Category-First→答案索引验证	+3.75 分

规则明确的任务（T1、T3）多轮重试收益很低——模型第一轮就答对了；复杂任务（T4、T7）多轮重试是主要的准确率提升来源。

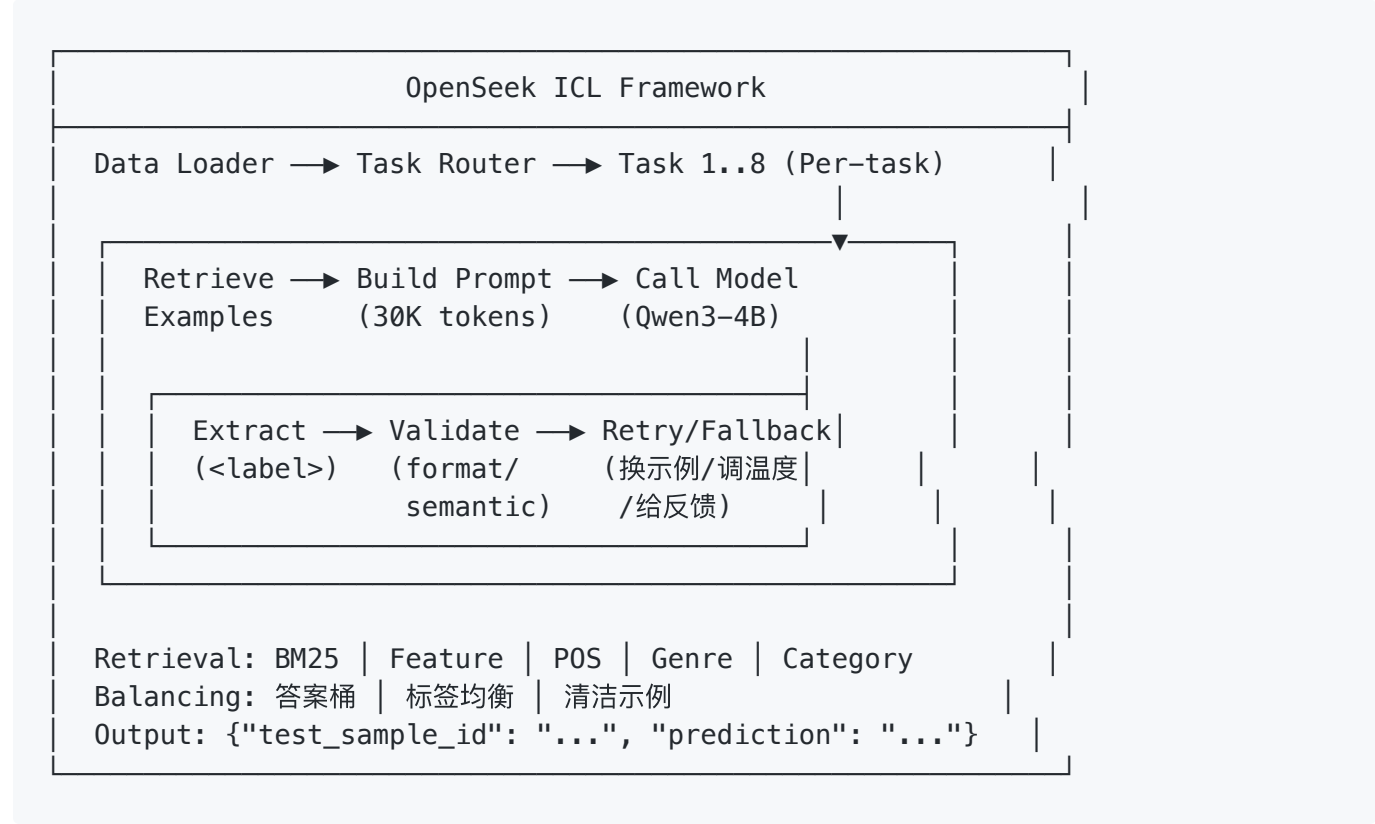
四、最终结果

任务	满分	最优得分	本版本准确率
T1 最近整数对	12.5	12.5	99.2%
T2 词性计数	12.5	10.15	81%
T3 Collatz 变换	12.5	12.5	99.8%
T4 字符串拼接	12.5	12.5	95.6%
T5 情感分类	12.5	10.73	~86%
T6 NLI 推理	12.5	8.75	~70%
T7 Jeopardy 问答	12.5	6.0	~43%
T8 代码生成	12.5	0.48	3%

五、未来方向

- 语义检索增强：引入 MiniLM 与 BM25 进行 RRF 融合，提升 T6 和 T7 的语义相关性检索。
- 自适应示例选择：根据模型对每个测试样本的置信度，动态调整 ICL 示例的选择策略。
- 更强的验证规则：T8 引入 Triton 语法检查。

附录 A：系统架构



本报告基于 *FlagOS Long-Context ICL Annotation* 竞赛的实际实验数据编写，所有结果均在 *Qwen3-4B* 模型上复现。